ZEBA ACADEMY / zeba.academy / Instructor – Sufyan bin Uzayr **Lab** **Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards**

Practicum 01: Empirical Parallel Computing

Duration: 90 Minutes **Format:** Solo In-Class Lab **Weight:** 5% of Course Grade

STUDENT NAME	STUDENT ID	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
Shynbay Daniyal	**230103158**	**AMD Ryzen 5 5625U (ASUS Vivobook 16 M1605YA)**	**Windows 11 Pro 64-bit (OS Build 26200.9445)**

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

15 Minutes | 20 Points Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches | macOS: sysctl -a | grep machdep.cpu | Windows: Run wmic cpu get /format:list and inspectTask Manager > Performance > CPU.

Hardware Property	Local Measured Value	Hardware Property	
CPU Model & Architecture	*AMD Ryzen 5 5625U (Zen 3 / x86_64)*	**Base & Max Boost Clock**	*2.30 GHz / 4.30 GHz*
Physical Hardware Cores	*6 Cores*	**Logical Cores (SMT/Threads)**	*12 Logical Processors*
L1 Data / Instruction Cache	*L1d: 32 KB/core (192 KB) | L1i: 32 KB/core (192 KB)*	**L2 Cache (Per Core / Total)**	*512 KB per core / 3.0 MB Total*
L3 Shared Last-Level Cache	*16.0 MB (Unified shared pool)*	**SIMD Vector Extensions**	*AVX, AVX2, FMA3, SSE4.2*

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

SISD (Single Instruction, Single Data): Occurs during single-threaded scalar code execution (e.g., standard OS processes, single-thread loop iterations). A single instruction stream executes sequentially on single scalar 64-bit registers without SIMD or thread-level parallelism.
SIMD (Single Instruction, Multiple Data): Executed by the Zen 3 core's vector execution units using AVX2 / FMA3 extensions. A single instruction (e.g., _mm256_fmadd_ps) simultaneously executes arithmetic operations on 8 single-precision (32-bit) or 4 double-precision (64-bit) floats across 256-bit YMM registers in a single clock cycle.
MIMD (Multiple Instruction, Multiple Data): The native operating mode of the 6 physical cores and 12 hardware threads managed by the Windows 11 scheduler. Multiple independent hardware cores execute separate instruction streams (different PCs) on distinct memory locations across DRAM and the shared 16 MB L3 cache simultaneously.

35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: *[e.g., Prime Sieve to 5,000,000 / Matrix Multiply]*			**Language & Runtime:** *[e.g., C OpenMP / Python multiprocessing / Go]*			
Threads (N)	**Run 1 (s)**	**Run 2 (s)**	**Run 3 (s)**	**Avg Time TN (s)**	**Speedup SN = T1/TN**	
N = 1 (Baseline)	**1,520**	**1,507**	**1,509**	**1,512**	1.00×	100.0%

N = 2	1,506	1,534	1,520	1,520	1,00 x	49,8 %
N = 4	1,025	0,848	1,230	1,034	1,46 x	36,6 %
N = 8	0,420	0,491	0,484	0,465	3,25 x	40,7 %
N = 16	0,308	0,303	0,317	0,309	4,89 x	30,6 %
N = 32	0,229	0,258	0,274	0,254	5,96 x	18,6 %

Q2.1: Scaling Saturation & Hardware Contention Analysis:

1. Initial Bottleneck (N = 2 N=2): At N = 2 N=2 , speedup stalled at 1.00 × 1.00× (efficiency dropped to 49.8 % 49.8%). This is caused by loop stride load-imbalance and mobile CPU frequency scaling: at N = 1 N=1 , a single active core boosts up to 4.30 GHz 4.30 GHz , whereas engaging multiple cores drops the sustained all-core boost clock to conserve the 15W TDP thermal envelope.
2. Multi-Core Scaling (N = 4 → 8 N=4→8): Scaling improves significantly between N = 4 N=4 (1.46 × 1.46×) and N = 8 N=8 (3.25 × 3.25×) as the workload distributes across all 6 physical Zen 3 cores.
3. Logical SMT & Oversubscription (N = 16 → 32 N=16→32): Efficiency degrades sharply at N = 16 N=16 (30.6 % 30.6%) and N = 32 N=32 (18.6 % 18.6%). The AMD Ryzen 5 5625U provides only 12 logical threads. Spawning 16 and 32 threads induces heavy CPU oversubscription, causing the Windows 11 kernel scheduler to spend excessive cycles on thread preemption, context switching, and thrashing the private L1/L2 caches.

25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: 10 × 1,000,000 = 10,000,000. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	
#1	1155967	8844033	#6	1313127	8686873
#2	1153323	8846677	#7	1165793	8834207
#3	1112383	8887617	#8	1148775	8851225
#4	1101546	8898454	#9	1162901	8837099
#5	1138911	8861089	#10	1090668	8909332

Q3.1: Read-Modify-Write (RMW) Window
In Java and at the machine instruction level, counter++ is a non-atomic three-step Read-Modify-Write (RMW) sequence:

1. getstatic / MOV (Load: read shared counter value from memory/cache into CPU register)
2. iadd / ADD (Modify: increment value in ALU)
3. putstatic / MOV (Store: write updated value back to shared memory)

When two Zen 3 cores execute concurrently without locks:

- Core 1 and Core 2 both read counter = 100 into registers simultaneously.
- Both compute 101 in their ALUs.
- Core 1 writes 101 to memory. Immediately after, Core 2 overwrites

Q3.2: The Synchronization Penalty
Wrap the counter in a Mutex/Lock. Record execution time:
Unlocked = 132 ms vs. Locked = 203 ms.
Why is race-free synchronized code substantially slower?

Why is race-free synchronized code substantially slower?

1. Serialization of Critical Section: The synchronized monitor enforces total mutual exclusion, forcing all 10 threads into a serial queue and neutralizing parallel core advantages.
2. Cache-Coherence Traffic (MOESI Protocol): Acquiring the monitor executes atomic instructions (LOCK CMPXCHG), broadcasting cache-invalidation signals across the Infinity Fabric to acquire the cache line in Modified/Exclusive state.

memory with 101. - Outcome: Two iterations occurred, but the global value increased by only 1. Core 2's write destroyed Core 1's work (Lost Update Hazard), causing an ~88% data loss rate across 10 threads.	3. Thread Parking & OS Latency: Thread contention causes JVM monitor inflation and OS-level thread blocking/unparking, resulting in microsecond-scale context switch overhead.

15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: p = 2 × (T1 - T2) / T1 $$p = 2 \times \frac{1.520 - 1.506}{1.520} = 2 \times \frac{0.014}{1.520} = \mathbf{0.0184}$$ T1 = 1.520 s | T2 = 1.506 s Calculated p = 0.0184 (decimal between 0.0 and 1.0)	**2. Amdahl Theoretical Speedup Ceiling** Formula: Smax = 1 / (1 - p) Sequential fraction (1 - p) = 1 − 0.0184 = 0.9816 Max Theoretical Speedup (N → ∞) =1 / 0.98161 = 1.02×
3. Amdahl 64-Core Projection Formula: S64 = 1 / [(1 - p) + (p / 64)] $$S_{64} = \frac{1}{0.9816 + \frac{0.0184}{64}} = \frac{1}{0.9816 + 0.0002875} = \frac{1}{0.98189} = \mathbf{1.02}\times$$ Projected Speedup on 64 Cores = 1.02 ×	**4. Gustafson Scaled Speedup (Weak Scaling)** Formula: SGustafson = (1 - p) + (p × 64) $$S_{\text{Gustafson}} = 0.9816 + (0.0184 \times 64) = 0.9816 + 1.1776 = \mathbf{2.16}\times$$ Projected Scaled Speedup (64× Problem) = 2.16 ×

Q4.1: Strong vs. Weak Scaling Analysis:

- *Amdahl's Law (Strong Scaling / Fixed Problem Size): Under fixed workload size, if the initial parallel fraction p p between 1 and 2 threads is small (1.84 % 1.84% due to thread start-up and frequency differences), Amdahl's law demonstrates that the massive serial fraction (1 − p = 98.16 % 1−p=98.16%) forms an insurmountable bottleneck. Regardless of adding 64 cores, speedup is asymptotically capped at 1.02 × 1.02× .*
- *Gustafson's Law (Weak Scaling / Scaled Problem Size): Gustafson assumes the problem size scales proportionally with the processing hardware (64 × 64× larger data). Because parallel work expands with the workload volume while serial dispatch overhead remains constant, Gustafson projects that even with low initial 2-thread speedup, scaled execution achieves greater throughput (2.16 × 2.16× scaled speedup), proving that weak scaling is far more resilient to serial bottlenecks than strong scaling.*

Submission Guidelines & Deliverables Checklist:

- Complete all 4 tables and technical reflection prompts directly on this document (or submit as an annotated Word / PDF).
- Attach a single terminal screenshot verifying Task 2 executing across active threads (e.g., htop, top, or Task Manager CPU pane).
- Upload the final .docx / PDF to the course portal before the conclusion of the 90-minute lab period.